

Cairo University

**Faculty of Computers and Artificial
Intelligence**



CS251

Introduction to Software Engineering

Hasala

Software Design Specifications

Version 1.0

ID	Name	Email	Mobile
20240114	Basem Mohamed abdelabor	20240114@stud.fci-cu.edu.eg	+201151930659
20230162	Zeyad Mustafa Ramadan Amin	20230162@stud.fci-cu.edu.eg	+201156597227
20240306	Abdelrahman Tarek Mahmoud	20240306@stud.fci-cu.edu.eg	+201101561323
20240830	Mohammed Foad Sayed	20240830@stud.fci-cu.edu.eg	+201146633775

CS251: Elfama 7aga

Project: budgetwise



Software Design Specification

April 2026

Contents

Team	3
Document Purpose and Audience	3
System Models	3
I. Architecture Diagram	3
II. Class Diagram(s)	4
III. Class Descriptions	5
IV. Sequence diagrams	6
Class - Sequence Usage Table	7
V. State Diagram	8
VI. SOLID Principles	8
VII. Design Patterns	8
Tools	8
Ownership Report	8

CS251: Elfama 7aga

Project: budgetwise



Software Design Specification

Team

ID	Name	Email	Mobile
20240114	Basem Mohamed abdelsabor	20240114@stud.fci-cu.edu.eg	+201151930659
20230162	Zeyad Mustafa Ramadan Amin	20230162@stud.fci-cu.edu.eg	+201156597227
20240306	Abdelrahman Tarek Mahmoud	20240306@stud.fci-cu.edu.eg	+201101561323
20240830	Mohammed Foad Sayed	20240830@stud.fci-cu.edu.eg	+201146633775

Document Purpose and Audience

Document purpose:

This document explains the design and implementation of the software system. It helps make the system structure and workflow clear so that development is easier and more organized. It ensures that all parts of the system are well defined and nothing is missed during implementation. It also serves as a guide for building a professional and well-structured software system.

Target Audience:

- 1-Project Manager: to plan, organize, and monitor the project activities based on the defined requirements.
- 2- Software Developers: to understand the required system functions and implement them correctly.
- 3- Client / Product Owner: to ensure that the system addresses the intended needs and includes the required features.
- 4- End Users: to understand the main capabilities of the system and how it is intended to support personal financial management



Software Design Specification

System Models

I. Architecture Diagram

Software Architecture: Layered Architecture with MVC in the Presentation Layer

1. Architecture Overview

- The system is designed using a **Layered Architecture**, where the system is divided into clear layers based on responsibility.
- The **backend follows the MVC (Model–View–Controller) architecture**, while the **frontend is built using vanilla HTML, CSS, and JavaScript with a modular structure**.

2. System Architecture Layers

The system is designed using a **Layered Architecture**, which separates the application into different layers, each with a specific responsibility:

- **Presentation Layer (Frontend):**
Built with HTML, CSS, and JavaScript. It provides the user interface and handles user interactions. The frontend communicates with the backend through API requests.
- **Business Logic Layer (Backend - MVC):**
Implements the MVC architecture:
 - **Model:** Manages data and business rules.
 - **Controller:** Handles requests and processes application logic.
 - **View:** Returns data responses (typically JSON) to the frontend.
- **Data Access Layer:**
Responsible for interacting with the database and performing CRUD operations.
- **Database Layer:**
Stores all persistent data such as users, transactions, and budgets

CS251: Elfama 7aga

Project: budgetwise



Software Design Specification

Presentation Layer (Frontend)

- Built using **HTML, CSS, and JavaScript (no frameworks)**.
- Uses a **modular feature-based structure** to organize code.
- Responsible for:
 - User interface (UI)
 - User interactions
 - Sending requests to the backend

Application / Business Logic Layer (Backend MVC)

- Implements the **MVC architecture**:
 - **Model**: Represents data and business rules (e.g., users, transactions, budgets)
 - **Controller**: Handles incoming requests, processes logic, and communicates with models
 - **View**: Returns responses (typically JSON for frontend consumption in web applications)
- This layer contains the core application logic such as:
 - budgeting calculations
 - transaction processing
 - validation rules

Data Access Layer

- Handles communication with the database.
- Provides CRUD operations (Create, Read, Update, Delete).
- Isolated from controllers to maintain separation of concerns.

Database Layer

- Stores persistent data such as:
 - user accounts
 - transactions
 - budgets



Software Design Specification

3. Frontend Structure (Modular Approach)

The frontend does not use a framework or strict MVC. Instead, it follows a **modular JavaScript structure**, where code is split into feature-based modules such as:

- `transactions.js` → handles transaction-related UI logic
- `budget.js` → handles budget UI behavior
- `storage.js` → handles data communication with backend
- `ui.js` → updates DOM elements

This improves maintainability and code organization.

4. Backend MVC Structure

The backend strictly follows MVC principles:

- **Model**
 - Defines system data structures and interacts with the database.
 - Example: User model, Transaction model, Budget model.
- **Controller**
 - Receives HTTP requests from the frontend.
 - Processes business logic using models.
 - Returns responses (usually JSON).
- **View**
 - In this system, the “View” is not server-rendered HTML.
 - Instead, it is represented by **API responses (JSON data)** consumed by the frontend.



Software Design Specification

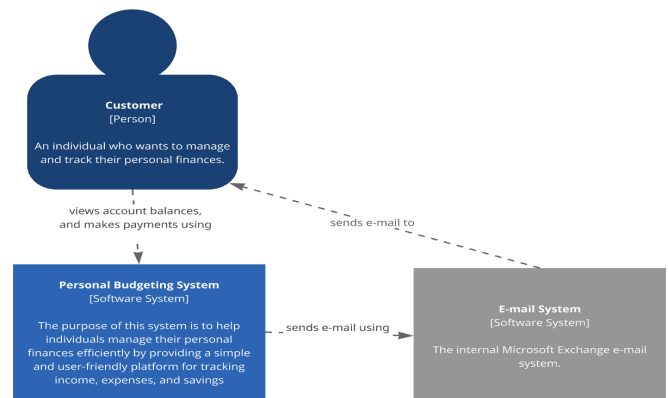
5. Why This Architecture is Suitable

- **Clear separation of responsibilities**
 - Frontend handles UI only
 - Backend handles logic and data processing
- **Maintainability**
 - Backend MVC ensures structured and organized server-side code.
- **Scalability**
 - New features can be added easily in either frontend modules or backend controllers/models.
- **Flexibility**
 - Frontend and backend are decoupled and communicate via APIs.
- **Industry relevance**
 - MVC is widely used in backend frameworks.
 - Modular JavaScript is a standard approach for vanilla frontend development.

C4 Notation:

1-System context diagram

hows the system as a whole and how it interacts with external users and systems.



System Context Diagram for Budgeting Banking System
The system context diagram for the Budgeting Banking System

CS251: Elfama 7aga

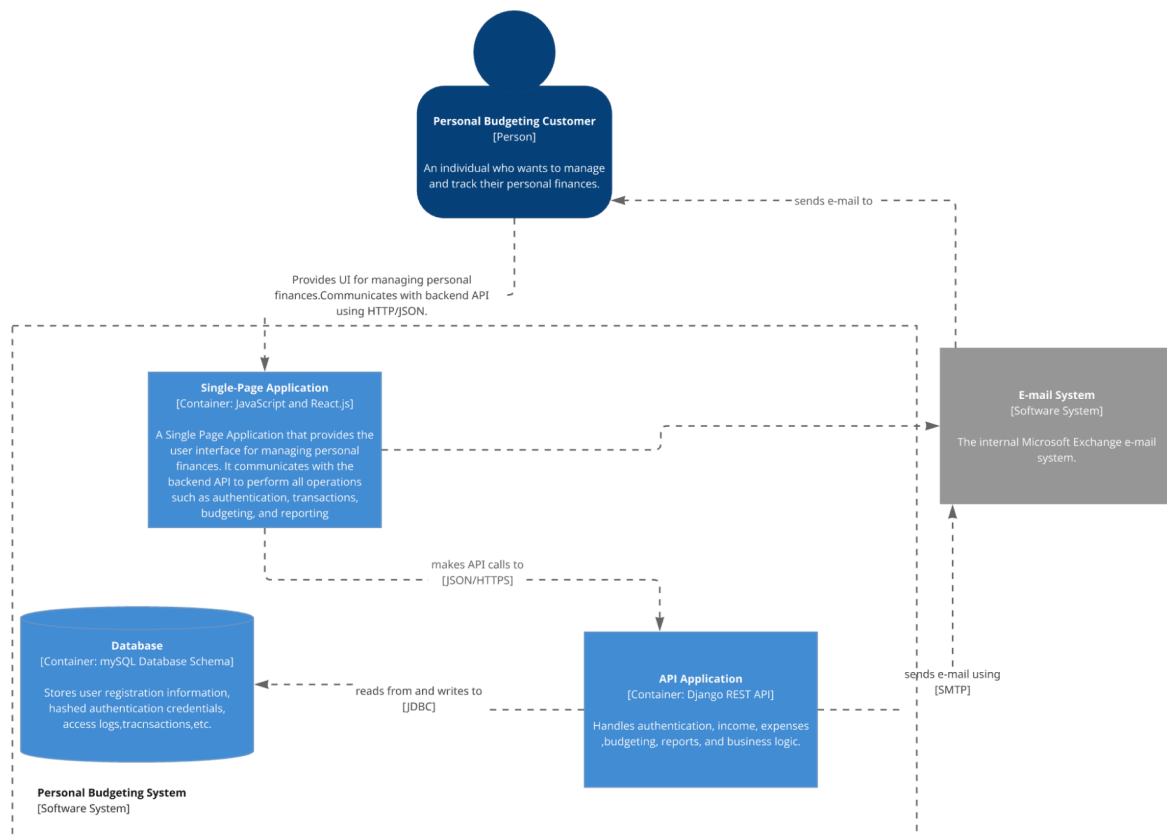
Project: budgetwise



Software Design Specification

2 -Container diagram

Breaks the system into major parts such as web apps, backend services, and databases.



Container Diagram for Budgeting Banking System
The container diagram for the Budgeting Banking System
Last modified: 2022-02-10

CS251: Elfama 7aga

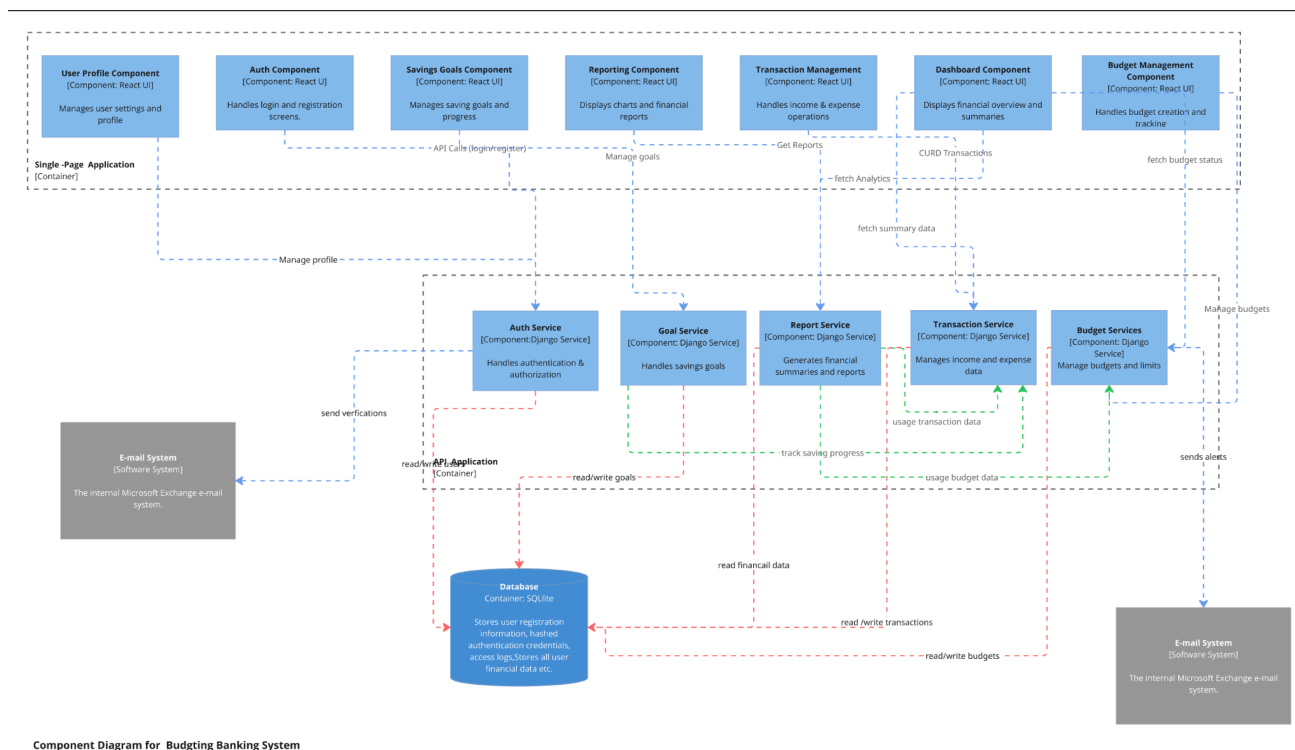
Project: budgetwise



Software Design Specification

3- Component diagram

Describes the internal structure of each container, showing modules and their relationships



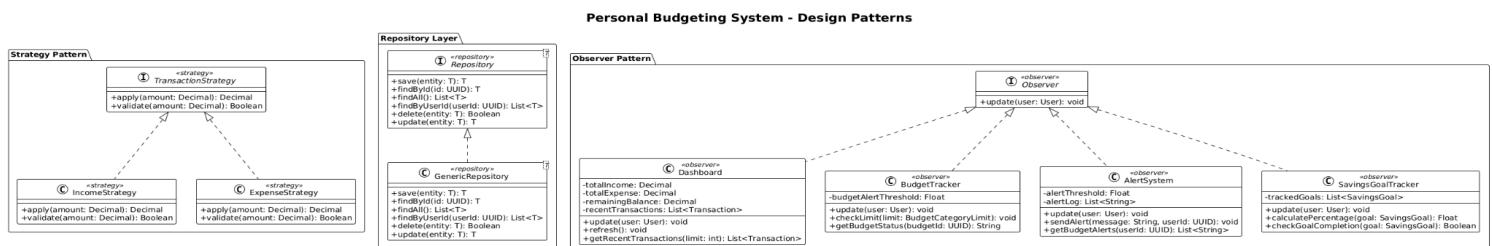
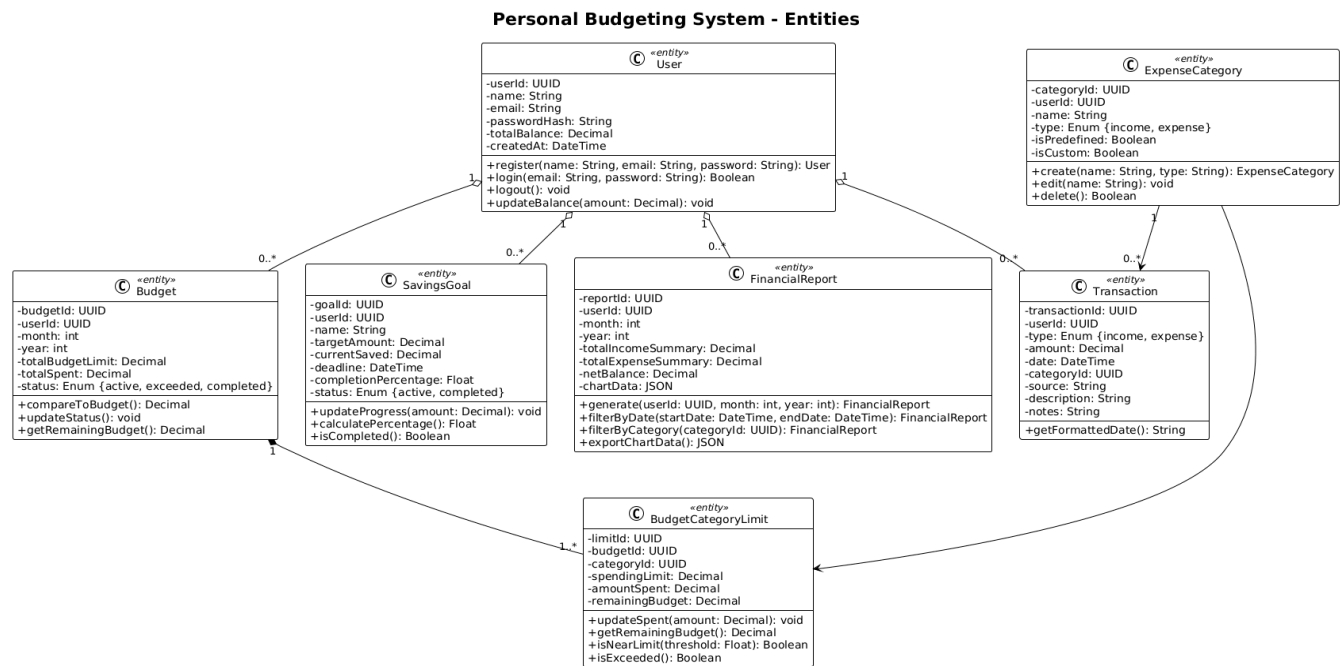
CS251: Elfama 7aga

Project: budgetwise



Software Design Specification

II. Class Diagram(s)



a

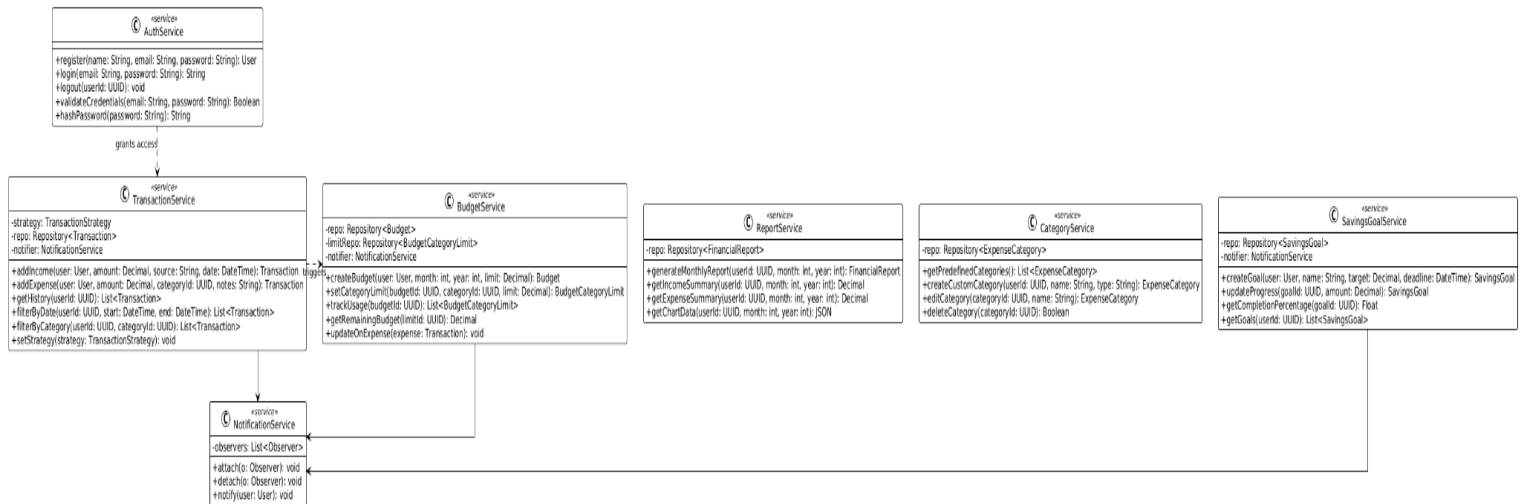
CS251: Elfama 7aga

Project: budgetwise

Software Design Specification



Personal Budgeting System - Service Layer (Grid View)





Software Design Specification

III. Class Descriptions

Class ID	Class Name	Description & Responsibility
1.	User	Represents a registered user in the system. Stores personal credentials (name, email, hashed password) and tracks the user's total financial balance
2.	Transaction	The core financial record of the system. Represents either an income or an expense entry made by a user. Holds the amount, date, category reference, income source (for income transactions), a description, and optional notes (for expenses). Every financial change in the system is stored as a Transaction.
3.	ExpenseCategory	Represents a label used to classify transactions. Can be either a system-provided predefined category or a user-created custom one. Supports both income and expense types via the <code>type</code> enum. Users can create, edit, or delete their custom categories, while predefined ones remain fixed.
4.	Budget	Represents a monthly spending plan created by a user. Holds the total budget limit for a specific month and year, tracks how much has been spent, and maintains a status (active, exceeded, or completed). Automatically compares actual spending against the defined limit and updates its status when new expenses are recorded.
5.	BudgetCategoryLimit	A child entity of Budget that defines the spending cap for a specific category within a budget. Tracks how much has been spent in that category, how much remains, and whether the limit has been exceeded or is close to being exceeded. Updated automatically whenever a relevant expense transaction is added.



Software Design Specification

Class ID	Class Name	Description & Responsibility
1.	SavingsGoal	Represents a financial target a user wants to reach. Stores the goal name, target amount, current saved amount, an optional deadline, and the completion percentage. Tracks progress over time and marks itself as completed when the target amount is reached.
2.	FinancialReport	A monthly summary report generated for a user. Aggregates total income, total expenses, and net balance for a given month and year. Supports filtering by date range and category, and exports chart-ready data (JSON) for visual representation in graphs and charts on the frontend
3.	TransactionStrategy	(interface) Defines the contract for processing a financial transaction. Any class implementing this interface must provide an <code>apply()</code> method that computes the effect of an amount on the user's balance, and a <code>validate()</code> method that checks whether the amount is valid before processing.
4.	IncomeStrategy	A concrete implementation of <code>TransactionStrategy</code> for expense transactions. Its <code>apply()</code> method subtracts the given amount from the user's balance. Used by <code>TransactionService</code> when the user records an expense entry.
5.	Repository<T>	(interface) A generic data-access contract that any repository class must implement. Defines the standard CRUD operations: <code>save()</code> , <code>findById()</code> , <code>findAll()</code> , <code>findByIdUser()</code> , <code>update()</code> , and <code>delete()</code> . Keeps the service layer decoupled from any specific database technology.
	GenericRepository<T>	The concrete implementation of the <code>Repository<T></code> interface. Handles the actual database interaction for any entity type <code>T</code> . Because it is generic, a single implementation serves all entities (User, Transaction, Budget, etc.) without duplicating data-access code.
	Observer	(interface) Defines the contract for any class that wants to be notified when the user's financial state changes. Any implementing class must provide an <code>update(user)</code> method that reacts to the change appropriately.
	Dashboard	An observer that maintains the live overview of the user's finances. Upon <code>update()</code> , it recalculates and refreshes total income, total expenses, remaining balance, and the list of recent transactions displayed on the home screen

CS251: Elfama 7aga

Project: budgetwise



Software Design Specification

Class ID	Class Name	Description & Responsibility
	BudgetTracker	An observer that monitors budget consumption after each transaction. Upon <code>update()</code> , it checks each <code>BudgetCategoryLimit</code> to see if spending is approaching or has exceeded the defined cap, and coordinates with <code>AlertSystem</code> when a threshold is breached.
	AlertSystem	An observer responsible for generating and logging alerts. When notified, it checks whether any budget limit is near or exceeded and fires a <code>sendAlert()</code> message to the user. Maintains an alert log so past warnings can be reviewed.
	SavingsGoalTracker	An observer that watches the progress of the user's savings goals. After each financial update, it recalculates the completion percentage of each active goal and marks goals as completed when the target is reached.
	AuthService	Handles all user authentication flows. Responsible for registering new accounts, validating login credentials, hashing passwords before storage, and managing logout. It is the entry point to the system for any new or returning user.
	NotificationService	The subject (publisher) in the Observer pattern. Maintains a list of registered observers and broadcasts a <code>notify(user)</code> call to all of them whenever a financially significant event occurs (a transaction is added, a budget is updated, etc.). Observers can be attached or detached dynamically.
	TransactionService	The central service for all financial transaction operations. Uses the Strategy pattern to decide whether to apply income or expense logic. After processing a transaction, it saves it via the repository and triggers <code>NotificationService</code> to update all observers. Also provides history retrieval and filtering by date or category.
	BudgetService	Manages the full lifecycle of budgets. Allows users to create a monthly budget, define per-category spending limits, and track usage. After every expense transaction, it calls <code>updateOnExpense()</code> to update the relevant <code>BudgetCategoryLimit</code> and recalculate the budget status (R5.1–R6.3).

CS251: Elfama 7aga

Project: budgetwise



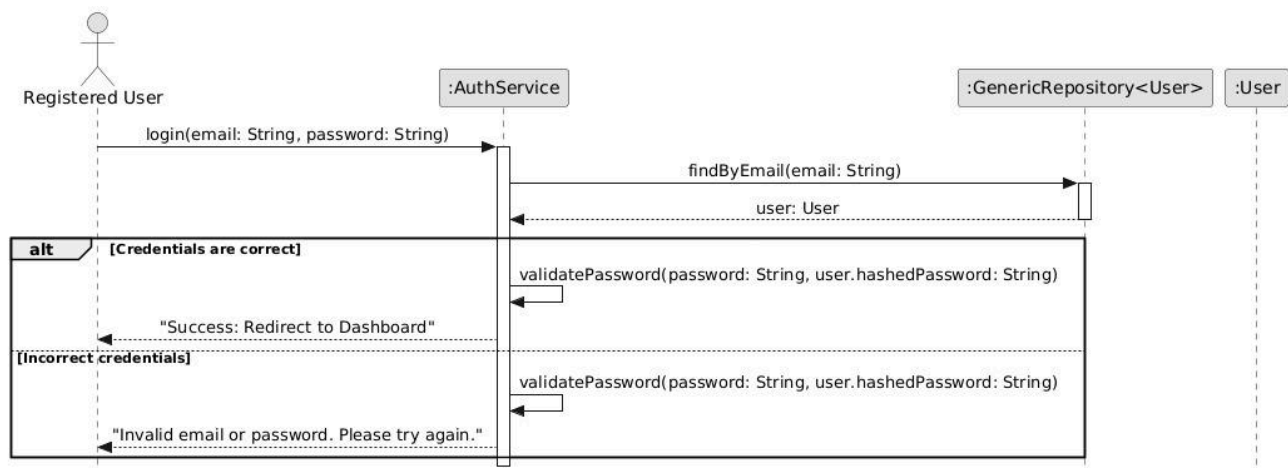
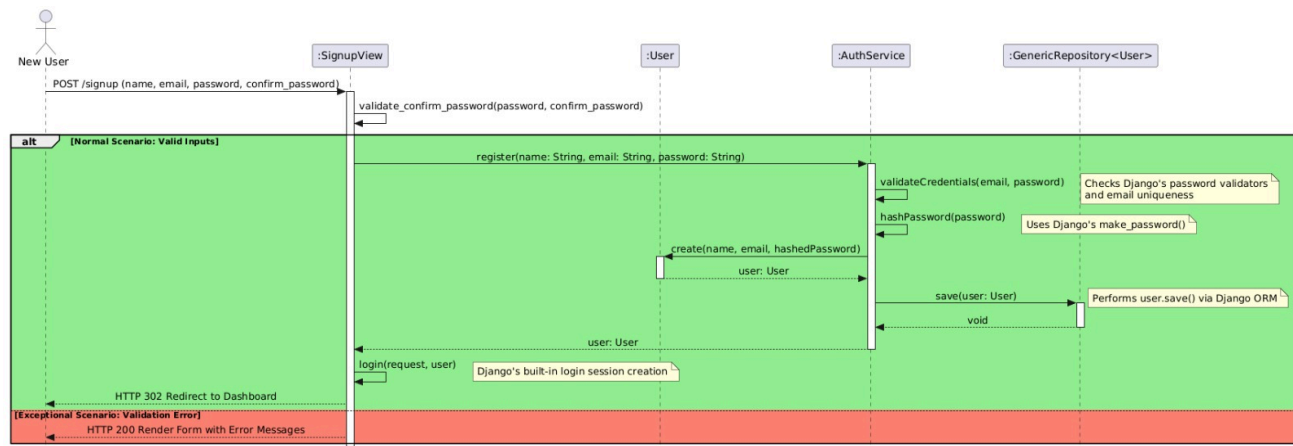
Software Design Specification

Class ID	Class Name	Description & Responsibility	SavingsGoal
	ReportService	Responsible for generating monthly financial reports. Aggregates income and expense data for a given user, month, and year, computes the net balance, and prepares chart data for visual display. Serves all requirements under the financial reporting feature	
	CategoryService	Manages expense and income categories. Provides the list of predefined system categories, and allows users to create, rename, or delete their own custom categories. Ensures that predefined categories cannot be deleted	
	SavingsGoalService	Handles the creation and progress tracking of savings goals. Allows users to define a target amount and deadline, record contributions toward the goal, and query the current completion percentage. Notifies observers when a goal is updated or completed.	



Software Design Specification

IV. Sequence diagrams

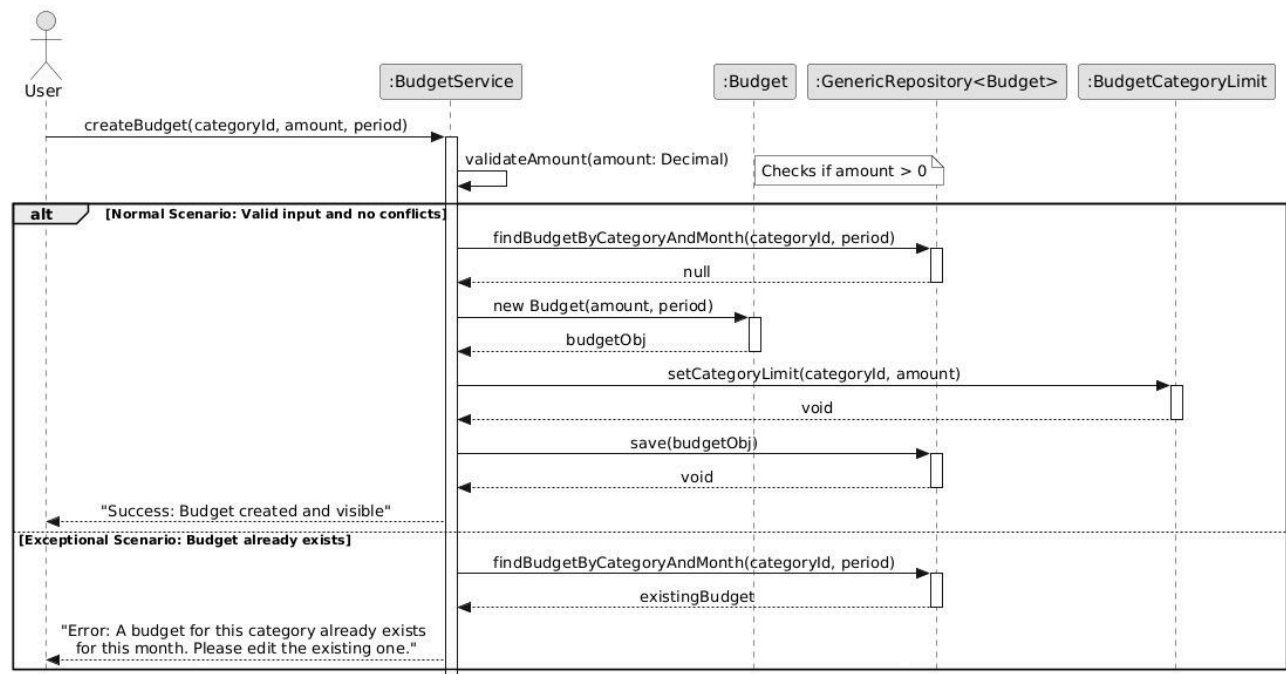
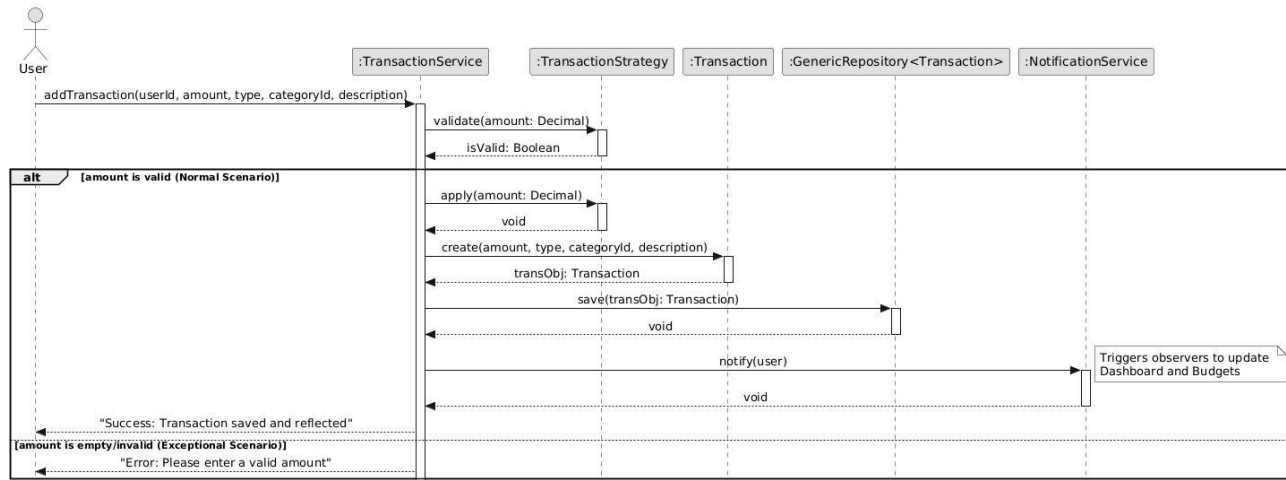


CS251: Elfama 7aga

Project: budgetwise



Software Design Specification

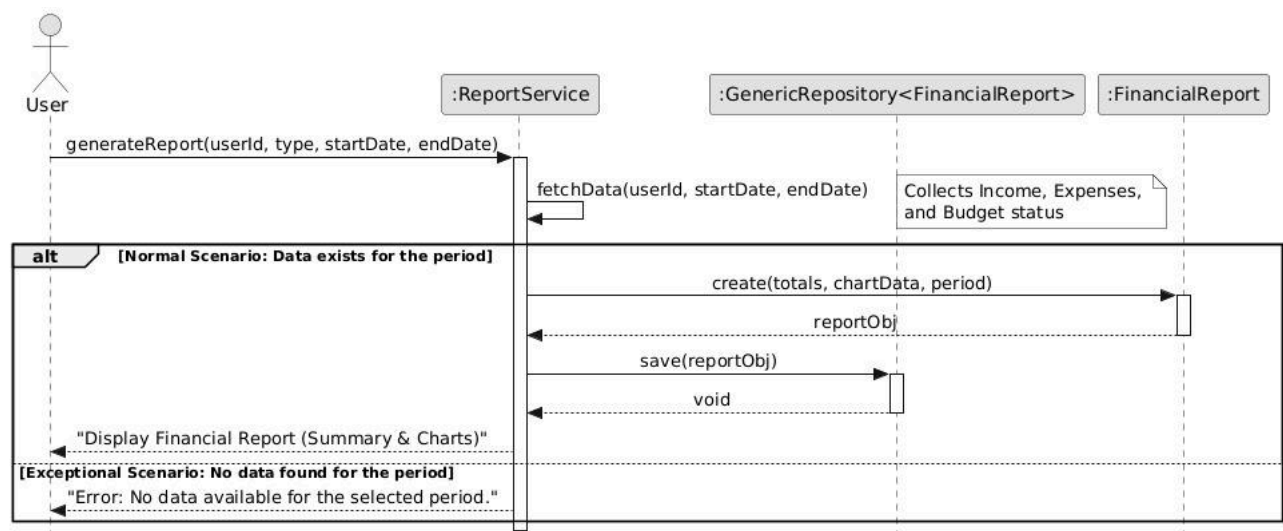
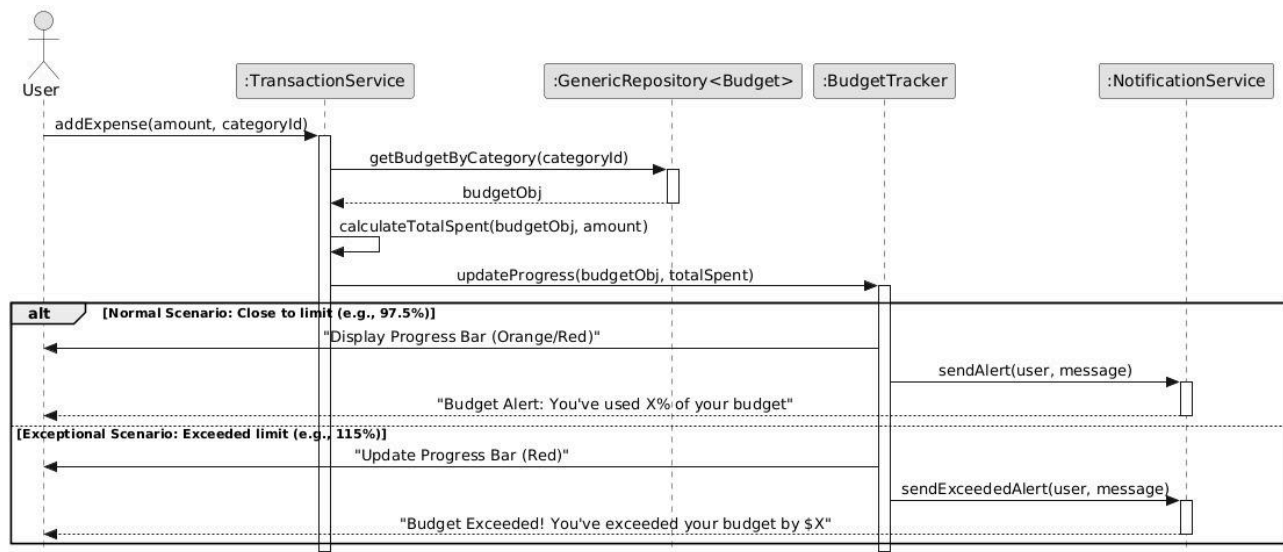


CS251: Elfama 7aga

Project: budgetwise

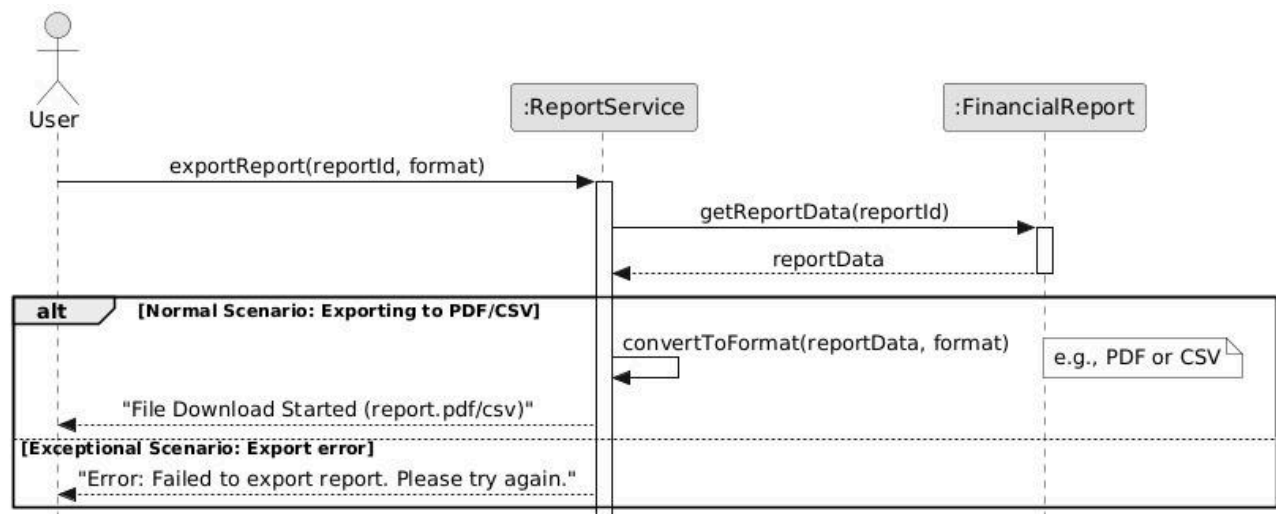


Software Design Specification





Software Design Specification



Class - Sequence Usage Table

#	Sequence Diagram (User Story)	Classes Used	All Methods Used
1	User Sign-Up	AuthService User GenericRepository	<code>register(name, email, password)</code> <code>validateCredentials(email, password)</code> <code>hashPassword(password)</code> <code>User.register(...)</code> <code>save(user)</code>

CS251: Elfama 7aga

Project: budgetwise



Software Design Specification

2	Log In	AuthService GenericRepository	<code>login(email, password)</code> <code>validatePassword(password, hashedPassword)</code> <code>findByEmail(email)</code>
3	Add Transaction	TransactionService TransactionStrategy Transaction GenericRepository NotificationService	<code>addTransaction(userId, amount, type, categoryId, description)</code> <code>validate(amount)</code> <code>apply(amount)</code> <code>create(amount, type, categoryId, description)</code> <code>save(transaction)</code> <code>notify(user)</code>
4	Create Budget	BudgetService GenericRepository Budget BudgetCategoryLimit	<code>createBudget(categoryId, amount, period)</code> <code>validateAmount(amount)</code> <code>findBudgetByCategoryAndMonth(categoryId, period)</code> <code>save(budget)</code> <code>new Budget(amount, period)</code> <code>setCategoryLimit(categoryId, amount)</code>



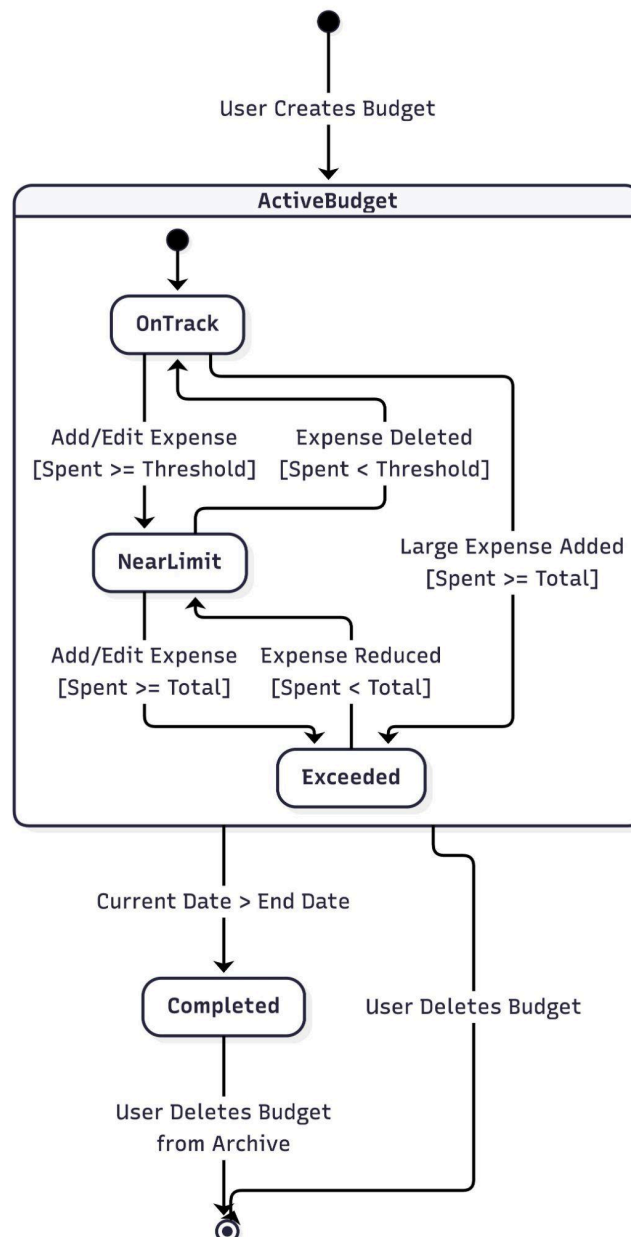
Software Design Specification

5	Budget Alert	TransactionService GenericRepository BudgetTracker NotificationService	<pre>addExpense(amount, categoryId) calculateTotalSpent(budget, amount) getBudgetByCategory(categoryId) updateProgress(budget, totalSpent) sendAlert(user, message) sendExceededAlert(user, message)</pre>
6	Generate Report	ReportService FinancialReport GenericRepository	<pre>generateReport(userId, type, start, end) fetchData(userId, start, end) create(totals, chartData, period) save(report)</pre>
7	Export Report	ReportService FinancialReport	<pre>exportReport(reportId, format) convertToFormat(data, format) getReportData(reportId)</pre>



Software Design Specification

V. State Diagram

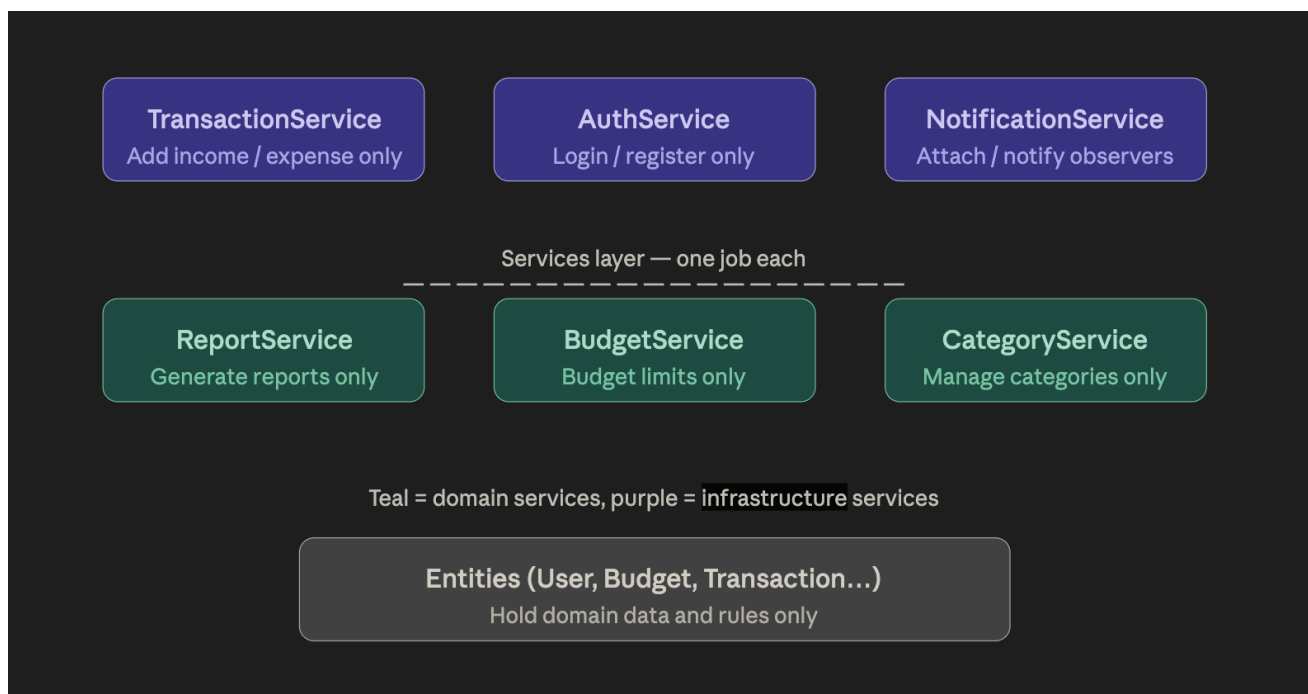




Software Design Specification

VI. SOLID Principles

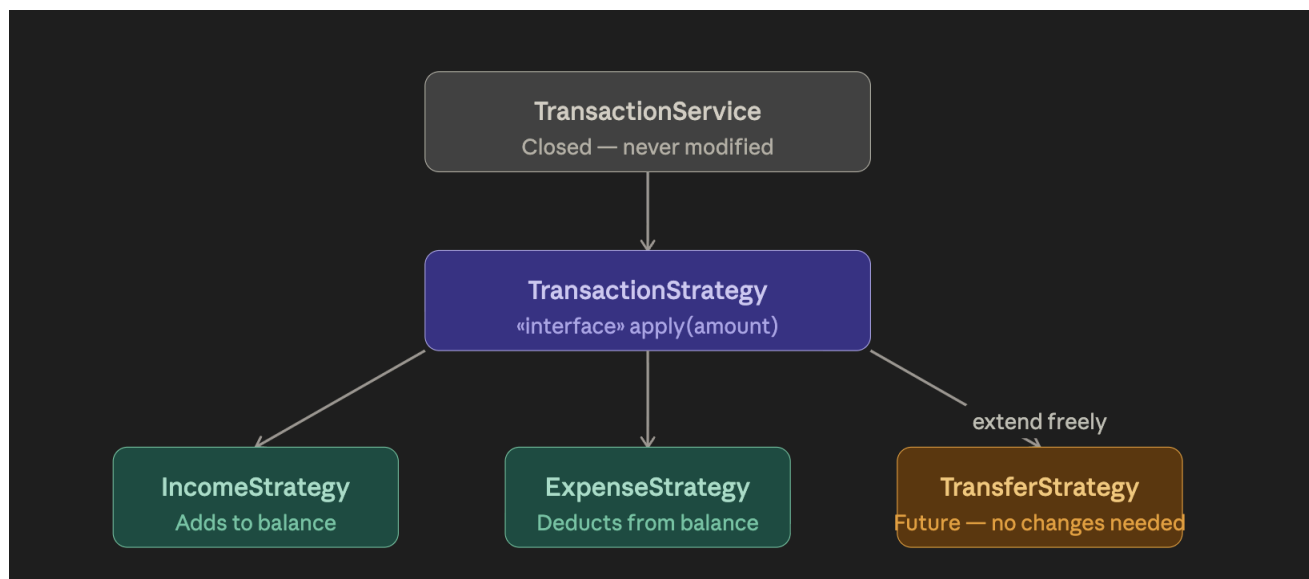
- **S — Single Responsibility Principle** is the most visibly applied principle. Every class has one clear job: **TransactionService** handles transaction logic only, **NotificationService** handles observer management only, **AuthService** handles authentication only, and each entity just holds its data and domain rules. No class tries to do two jobs





Software Design Specification

- **O — Open/Closed Principle** is applied cleanly through the Strategy Pattern.
`TransactionService` is open for extension (you can add new transaction types like `TransferStrategy` or `RefundStrategy`) but closed for modification — the service itself never changes, only the injected strategy does.

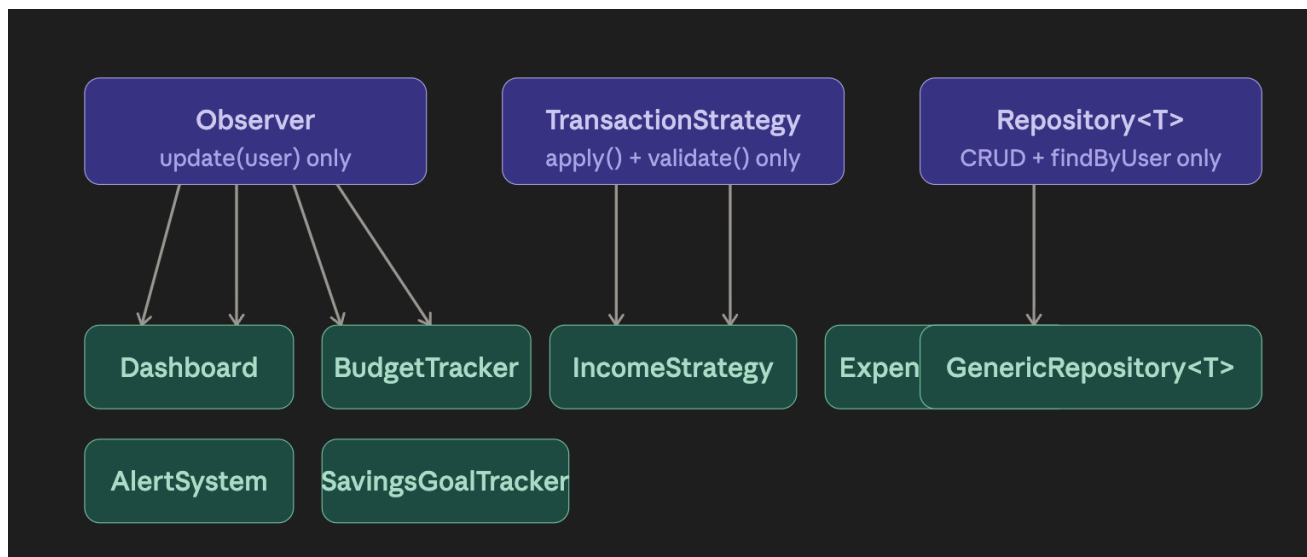


- **L — Liskov Substitution Principle** is upheld in both the Strategy and Observer patterns. Any concrete strategy (`IncomeStrategy`, `ExpenseStrategy`) can replace `TransactionStrategy` anywhere without breaking the caller. Any concrete observer (`Dashboard`, `AlertSystem`, `BudgetTracker`, `SavingsGoalTracker`) can substitute for `Observer` in the `NotificationService` list — same contract, different behavior



Software Design Specification

- **I — Interface Segregation Principle** is applied through targeted, lean interfaces. Rather than one fat interface, the design has three focused ones: **TransactionStrategy** (one method: **apply()**), **Observer** (one method: **update()**), and **Repository<T>** (only data-access methods). No class is forced to implement methods it doesn't need.



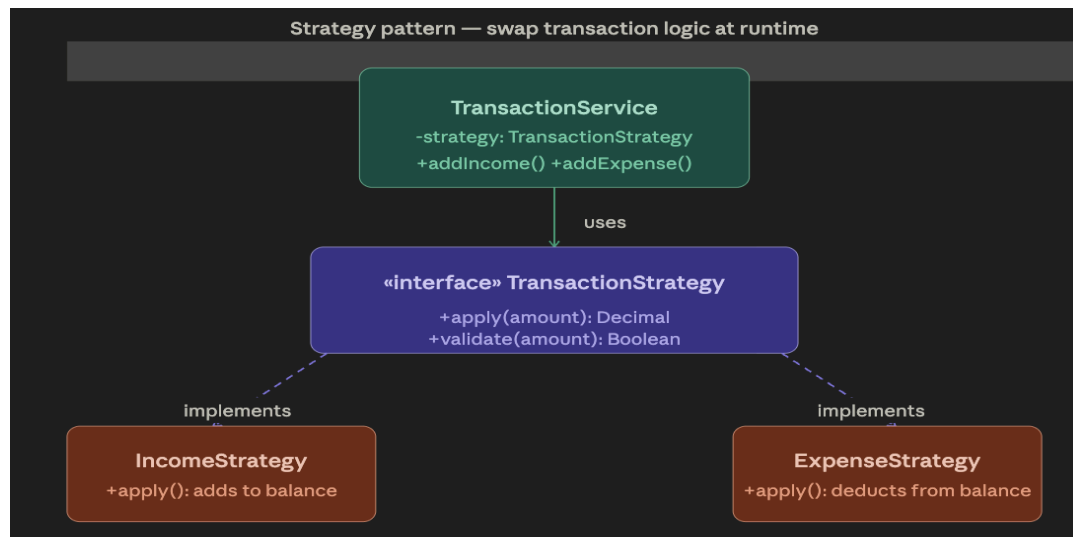
D — Dependency Inversion Principle is the backbone of how services connect to everything else. **TransactionService** depends on the **TransactionStrategy** interface, not on **IncomeStrategy** or **ExpenseStrategy** directly. It depends on **Repository<T>**, not **GenericRepository**. Services talk to abstractions, and the concrete implementations are injected from outside — making every service independently testable and swappable.



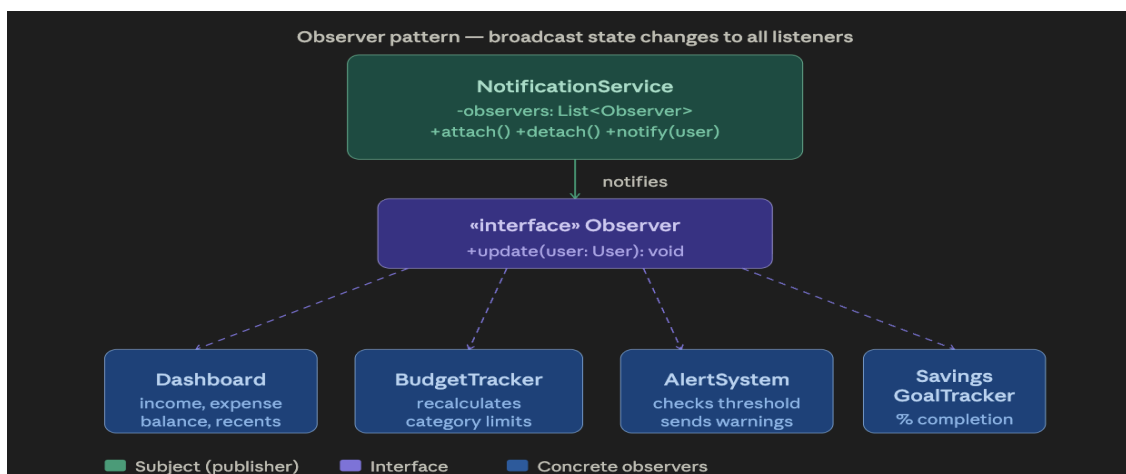
Software Design Specification

VII. Design Patterns

- **Pattern 1 — Strategy.** Used in **TransactionService** to swap between income and expense calculation logic at runtime, so adding a new transaction type never changes the service itself.



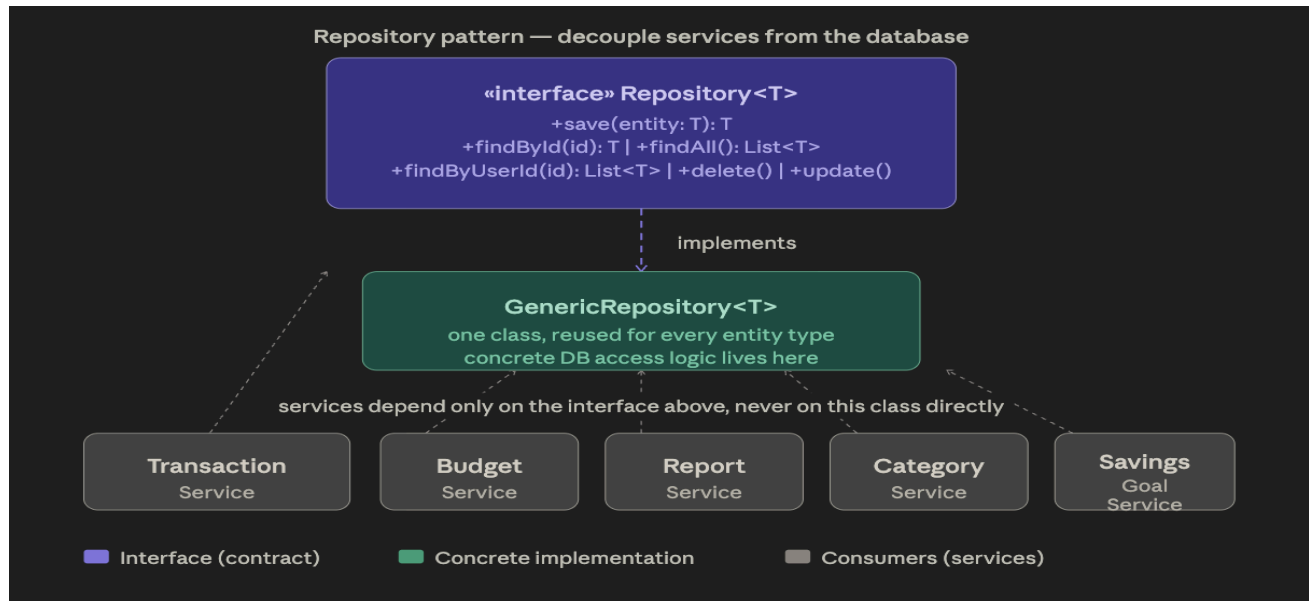
- **Pattern 2 — Observer.** **NotificationService** is the subject. Every time a transaction is added or a budget changes, it calls **notify(user)** and all attached observers react independently — dashboard refreshes, budget tracker recalculates, alert system checks thresholds, savings tracker updates progress.





Software Design Specification

Pattern 3 — Repository (Generic). A single `Repository<T>` interface abstracts all database access. Every service talks to this interface, never to a real database directly. `GenericRepository<T>` is the one concrete implementation, reused for every entity type — `Transaction`, `Budget`, `SavingsGoal`, etc.



CS251: Elfama 7aga

Project: budgetwise



Software Design Specification

Tools

- PlantUML, Miro, Claude, Mermaid.ai

Ownership Report

Item	Owners
Basem Mohamed Abdelsabor	Class Diagram(s), Class Description, SOLID Principles, Design Patterns
Zeyad Mostafa	Sequence Diagram, Sequence Description
Mohamed Foad	State Diagram
Abdelrahman Tarek	<i>Task 1 Report</i> , Architecture Diagram